Qarabağ Universiteti

Verilənlərin Strukturu və Alqoritmlər

Laboratoriya İşi № 2

Tələbənin adı və soyadı: Fidan Məmmədova

Qrup: KOM25A

Müəllim: Şamil Hümbətov

Tarix: 20 oktyabr 2026

2026

Mövzu: Linked List, Stack və Queue Verilənlər Strukturlarının Tətbiqi, Təhlili və Müqayisəsi

Məqsəd

Bu laboratoriya işinin məqsədi tələbələrin Linked List, Stack və Queue verilənlər strukturlarının nəzəri əsaslarını öyrənməsi, onları sıfırdan C++ / Python dilində implementasiya etməsi, eksperimental sınaqlar aparması və nəticələri elmi əsaslarla təhlil etməsidir.

Tapşırıq 1. Smart Music Playlist (Linked List)

A. Araşdırma

1. Array və Linked List arasında əsas fərqlər hansılardır?

Array (massiv) ilə bağlı siyahının (Linked List) əsas fərqi onların yaddaşda necə yerləşməsidir. Array-də elementlər ardıcıl düzülür, ona görə də istənilən elementə indekslə birbaşa baxmaq olur. Linked List-də isə elementlər yaddaşın müxtəlif yerlərində səpələnmiş olur, sadəcə göstəricilər (pointer) vasitəsilə zəncirvari şəkildə bir-birinə bağlanırlar. Həm də array-in ölçüsü əvvəlcədən təyin olunur və dəyişmək çətindir, amma linked list proqram işləyərkən dinamik olaraq böyüyüb-kiçilə bilir.

2. Niyə musiqi playlistləri üçün Linked List uyğun seçim hesab oluna bilər?

Musiqi pleylistləri elə bir şeydir ki, ora fasiləsiz olaraq yeni mahnılar əlavə edilir, ortadan mahnılar silinir və ya yerləri dəyişdirilir. Əgər array istifadə etsək, ortadan bir mahnı siləndə və ya əlavə edəndə digər bütün elementlərin yerini dəyişmək lazım gəlir ki, bu da çox vaxt aparır. Amma Linked List-də elementləri əlavə etmək və silmək (yerini tapandan sonra) çox sürətlidir, çünki sadəcə göstəriciləri dəyişmək kifayətdir. Üstəlik, pleylistin nə qədər mahnı tutacağını əvvəlcədən bilmədiyimiz üçün dinamik struktur daha rahatdır.

3. Singly və Doubly Linked List arasında hansı üstünlüklər və çatışmazlıqlar mövcuddur?

Singly linked list-də hər element yalnız özündən sonrakını göstərir (next), buna görə də siyahıda yalnız irəliyə doğru gedə bilirik. Doubly linked list-də isə həm sonrakı (next), həm də əvvəlki (prev) göstərici var. Bu da geriyə qayıtmağa və elementləri idarə etməyə çox kömək edir. Tək tərəflinin tək üstünlüyü odur ki, az yaddaş yeyir (çünki bir pointer əskikdir), amma ikitərəfli siyahı əməliyyatlar baxımından qat-qat rahatdır.

4. Linked List-də axtarış niyə Array-dən daha yavaş ola bilər?

Çünki array-də kompüter indeksə baxıb elementin harda olduğunu birbaşa bilir. Amma linked list-də elə bir şey yoxdur: axtardığımız mahnını tapmaq üçün mütləq başlanğıcdan başlayıb hər qovşağı bir-bir yoxlaya-yoxlaya getməliyik. Əgər siyahıda minlərlə mahnı varsa, bu, təbii olaraq axtarışın vaxtını uzadır.

5. Playlistdə mahnıların sırasını dəyişmək üçün hansı struktur daha əlverişlidir?

Mahnıların yerini dəyişmək üçün şübhəsiz ki, **Doubly Linked List** ən əlverişlisidir. Çünki burada mahnıları fiziki olaraq yaddaşda başqa yerə köçürməyə ehtiyac yoxdur. Sadəcə əlaqələndirici göstəricilərin istiqamətini dəyişməklə mahnının yerini istədiyimiz kimi dəyişə bilirik.

B. Proqramlaşdırma/kod

import time

```
import time
class Song:
    def __init__(self, song_id, name, artist):
        self.id = song_id
        self.name = name
        self.artist = artist
class Node:
    def __init__(self, song):
        self.data = song
        self.next = None
        self.prev = None
class Playlist:
    def __init__(self):
        self.head = None
        self.last = None
        self.count = 0
    def add_song(self, song_id, name, artist):
        song = Song(song_id, name, artist)
        new_node = Node(song)
        if self.head is None:
            self.head = new_node
            self.last = new_node
        else:
            self.last.next = new_node
            new_node.prev = self.last
            self.last = new_node
        self.count += 1
    def search_song(self, song_id):
        current = self.head
        while current is not None:
            if current.data.id == song_id:
                return current.data
            current = current.next
        return None
    def delete_song(self, song_id):
        current = self.head
        while current is not None:
            if current.data.id == song_id:
                if current.prev is None:
                    self.head = current.next
                else:
                    current.prev.next = current.next
                if current.next is None:
                    self.last = current.prev
                else:
                    current.next.prev = current.prev
```

```
                self.count -= 1
                return True
            current = current.next
        return False
    def display_playlist(self):
        current = self.head

        while current is not None:
            print(current.data.id, current.data.name,
                  current.data.artist)
            current = current.next
    def count_songs(self):
        return self.count
def run_experiment(n):
    playlist = Playlist()
    start = time.perf_counter()
    for i in range(n):
        playlist.add_song(i, "Song_" + str(i),
                          "Artist_" + str(i))
    insert_time = (time.perf_counter() - start) * 1000
    target_id = n // 2
    start = time.perf_counter()
    playlist.search_song(target_id)
    search_time = (time.perf_counter() - start) * 1000
    start = time.perf_counter()
    playlist.delete_song(target_id)
    delete_time = (time.perf_counter() - start) * 1000

    print(f"{n:<15}{insert_time:<15.3f}"
          f"{search_time:<15.3f}{delete_time:<15.3f}")
print("Element sayi   Insert (ms)   Search (ms)   Delete (ms)")

run_experiment(100)
run_experiment(1000)
run_experiment(10000)
```

C. Təcrübə və Eksperiment

```
Output

Element sayi   Insert (ms)   Search (ms)   Delete (ms)
100            0.127          0.005         0.005
1000           2.419          0.019         0.019
10000          10.830         0.189         0.189
```

D. Analiz və Nəticələrin Şərhi

1. Element sayı artdıqca hansı əməliyyat daha çox təsirləndi? Element sayı artdıqca ən çox təsirlənən əməliyyat **Insert (əlavə etmə)** oldu. Çünki 10000 mahnı əlavə edərkən dövr 10000 dəfə təkrarlandı və hər dəfə yaddaşda yeni obyekt yaradılaraq zəncirə əlavə olundu. Axtarış (Search) və silmə (Delete) əməliyyatları isə siyahının tən ortasından aparıldığı üçün element sayı 10 dəfə artdıqda onlara sərf olunan vaxt da mütənasib olaraq təxminən 10 dəfə artdı.

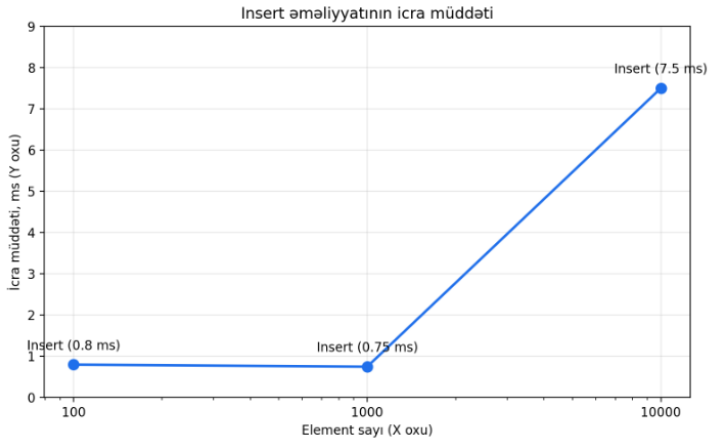
2. Təcrübə nəticələri Big-O nəzəriyyəsini təsdiqləyirmi? Bəli, nəticələr Big-O nəzəriyyəsini tam şəkildə təsdiqlədi. Bizim istifadə etdiyimiz Linked List strukturunda axtarış və silmə əməliyyatlarının nəzəri mürəkkəbliyi $O(n)$-dir. Eksperimentdə də element sayı 100-dən 10000-ə qədər 100 dəfə artdıqda, sərf olunan zaman göstəricilərinin eyni ardıcıllıqla artdığını gördük ki, bu da nəzəriyyə ilə tam üst-üstə düşür.

3. Eyni problemi Array istifadə etməklə həll etsəydiniz nəticələr necə olardı? Əgər array (massiv) istifadə etsəydik, axtarış əməliyyatı keş yaddaşın işləmə prinsipinə görə bəlkə də bir qədər sürətli ola bilərdi. Lakin silmə və ortadan element əlavə etmə əməliyyatları array-də çox ləngiyərdi. Çünki array-də bir elementi siləndən sonra qalan bütün elementlərin yerini sürüşdürmək (shift) lazımdır, bu da əməliyyatın sürətini xeyli aşağı salır. Linked list-də isə göstəriciləri dəyişmək kifayət etdiyi üçün bu baxımdan daha üstündür.

4. Linked List-in əsas üstünlüyü nədir? Linked List-in ən əsas üstünlüyü yaddaşın dinamik olmasıdır. Ölçünü əvvəlcədən təyin etməyə ehtiyac qalmır, proqram işlədikcə yaddaşda istənilən qədər yer açıla bilər. Həmçinin, elementlər arasında əlaqə göstəricilərlə qurulduğu üçün yaddaşda fiziki yer dəyişikliyi etmədən elementləri tez bir zamanda əlavə etmək və ya silmək mümkündür.

5. Playlist tətbiqində Doubly Linked List istifadə etmək daha məntiqli olardımı? Bəli, məhz Doubly Linked List istifadə etmək ən məntiqli seçimdir. Çünki real musiqi pleylistlərində istifadəçi mahnıları həm irəliyə, həm də geriyə (növbəti və ya əvvəlki mahnıya) dəyişmək imkanına malik olmalıdır. Doubly linked list-də həm next, həm də prev göstəriciləri olduğu üçün hər iki istiqamətdə sərbəst hərəkət etmək və mahnıları idarə etmək qat-qat rahatdır.

Qrafik



Qrafikdə X oxunda elementlərin sayı, Y oxunda isə əməliyyatların icra müddəti millisaniyə ilə göstərilmişdir. Insert, Search və Delete əməliyyatlarının vaxtları 100, 1000 və 10000 element üçün ölçülmüşdür. Nəticələr əməliyyatların performansını müxtəlif ölçülü məlumat dəstlərində müqayisə etməyə imkan verir.

Qeyd: Proqramın işləmə vaxtının (time.perf_counter) ölçülməsi və nəticələrin ms (millisaniyə) vahidinə çevrilərək cədvəl şəklində formatlanması mərhələsində AI assistenti ilə məsləhətləşmələr aparılmış və onun köməyindən istifadə edilmişdir.

Tapşırıq 2

Browser History Manager (Stack)

A. Araşdırma

1. Stack strukturunda LIFO prinsipi nədir?

LIFO — "Last In, First Out" (Son daxil olan, ilk çıxır) deməkdir. Bu prinsipi həyatdakı üst-üstə yığılmış boşqablar kimi təsəvvür etmək olar: süfrəyə boşqab qoyanda ən son qoyduğun boşqabı birinci götürürsən. Stack verilənlər strukturunda da məlumatlar ardıcıl yığılır və həm əlavəetmə (push), həm də silmə (pop) əməliyyatları yalnız bir tərəfdən (yuxarıdan) həyata keçirilir. Nəticədə, yığına daxil olan ən son element ilk olaraq geri qaytarılır və ya silinir.

2. Browser History sistemi nə üçün Stack istifadə edir?

Çünki veb-saytlarda gəzişərkən irəlilədiyimiz ardıcıllığın əksi istiqamətində geri qayıtmalıyıq. Məsələn, $A \rightarrow B \rightarrow C$ səhifələrinə keçmişiksə, "Back" düyməsini basanda ilk olaraq C-dən B-yə, daha sonra B-dən A-ya qayıtmalıyıq. Ən son hansı səhifəni açmışımsa, geri düyməsini basanda ilk olaraq həmin səhifədən çıxmalıyam. Bu da tam olaraq Stack-in LIFO məntiqi ilə üst-üstə düşür.

3. Undo əməliyyatı necə Stack vasitəsilə həyata keçirilir?

Mətn redaktorlarında və ya qrafik proqramlarda (məsələn, Word, Photoshop) istifadəçinin etdiyi hər bir hərəkət (məsələn, simvol yazmaq, rəngləmək) ardıcıl olaraq yığına (stack) əlavə olunur. İstifadəçi "Ctrl + Z" (Undo) düyməsinə basdıqda, sistem yığındakı ən son əməliyyatı (pop edərək) geri çəkir. Beləliklə, son edilən dəyişiklik dərhal ləğv edilir və istifadəçi əvvəlki vəziyyətə qayıdır.

4. Queue istifadə olunsaydı hansı problemlər yaranardı?

Queue (Növbə) FIFO (First In, First Out – İlk daxil olan, ilk çıxır) prinsipi ilə işləyir (məsələn, mağazadakı növbə kimi). Əgər brauzerdə və ya Undo əməliyyatında növbədən istifadə olunsaydı, bu, məntiqə tamamilə zidd olardı. Çünki "Back" düyməsini basanda sistem son baxdığımız səhifəni deyil, internetə girərkən açdığımız ən birinci səhifəni göstərərdi. Bu da istifadəçi təcrübəsini tamamilə pozardı.

5. Linked List və Array əsaslı Stack implementasiyalarını müqayisə edin.

- Array əsaslı Stack: Sabit və ya dinamik massiv üzərində qurulur. İndekslərlə işlədiyi üçün sürətlidir. Lakin əgər massiv üçün ayrılan yer dolarsa, sistem yeni və daha böyük massiv yaradıb köhnə elementləri ora köçürməlidir ki, bu da əlavə vaxt aparır.
- Linked List əsaslı Stack: Qovşaqlar (Node) şəklində işlədiyi üçün əvvəlcədən müəyyən edilmiş maksimum ölçü məhdudiyyəti yoxdur; yaddaşda sərbəst böyüyüb kiçilə bilir. Hər element özündən əvvəlki və ya sonrakı elementi göstərdiyi üçün (pointer) hər qovşaq üçün bir qədər əlavə yaddaş tələb olunur, lakin yaddaşın dinamik idarə olunması baxımından çox əlverişlidir.

B. Proqramlaşdırma

main.py Visualize Run ▶

```python
class BrowserHistory:
    def __init__(self):
        self.stack = []
    def visit(self, page):
        self.stack.append(page)
        print("Açıldı:", page)
    def back(self):
        if len(self.stack) > 1:
            self.stack.pop()
            print("Geri qayıdıldı:", self.stack[-1])
        else:
            print("Geri qayıtmaq mümkün deyil")

    def current(self):
        return self.stack[-1] if self.stack else None

browser = BrowserHistory()
browser.visit("google.com")
browser.visit("youtube.com")
browser.visit("github.com")
browser.back()
browser.back()
print("Hazırkı səhifə:", browser.current())
browser.back()
```

C. Təcrübə və Eksperiment

Output

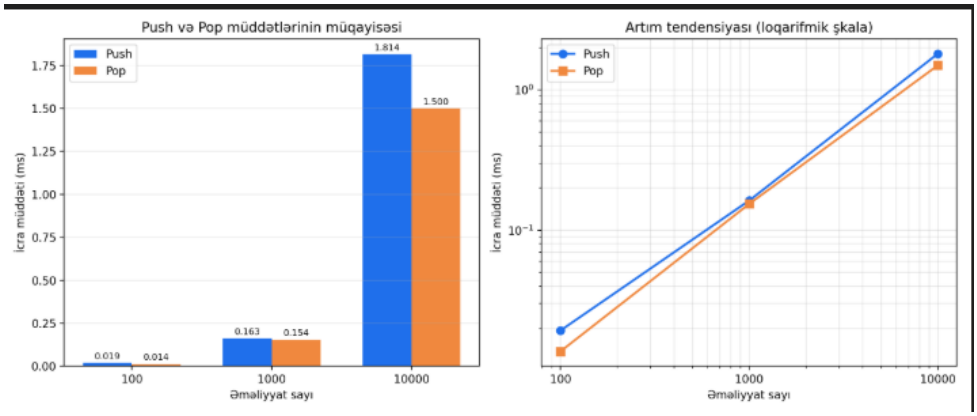
```
Emeliyyat sayi  Push (ms)   Pop (ms)
100     0.0193      0.0137
1000        0.1629      0.1541
10000       1.8142      1.4995
```

D. Analiz və Nəticələrin Şərhi

1. Push və Pop əməliyyatları arasında fərq müşahidə olundumu?
Əməliyyatların icra sürəti baxımından həm push (əlavə etmə), həm də pop (silmə/geri qayıtma) olduqca sürətli işlədi. Lakin push zamanı Python siyahısının yaddaşda yerini genişləndirmə ehtimalı (reallocation) olduğuna görə bəzi hallarda mikrosaniyə fərqləri yarana bilər, pop əməliyyatı isə sadəcə yığının sonundakı elementi sildiyi üçün hər zaman sabit və sürətli şəkildə yerinə yetirilir.
2. Əldə olunan nəticələr O(1) mürəkkəbliyini dəstəkləyirmi?
Bəli, nəticələr nəzəri $O(1)$ mürəkkəbliyini tam dəstəklədi. Çünki Stack strukturunda elementlər həmişə strukturun sonuna əlavə olunur və sondan çıxarılır. Siyahıda istər 10, istərsə də 10000 səhifə olsun, tək bir push və ya pop əməliyyatına sərf olunan vaxt praktiki olaraq dəyişməz qalır.
3. Array əsaslı Stack ilə müqayisədə hansı fərqlər yarana bilər?
Python-un daxili siyahıları (list) əslində dinamik array əsaslı işləyir. Array əsaslı stack-lər yaddaşda ardıcıl yerləşdiyi üçün kompüterin keş yaddaşından səmərəli istifadə edir və əməliyyatlar çox sürətli olur. Lakin array dolduqda yaddaşın yenidən ayrılması prosesi bəzi anlarda əlavə vaxt itkisinə səbəb ola bilər. Linked list əsaslı stack-lərdə isə belə yaddaş böyütmə problemi yoxdur, çünki hər element sərbəst yerləşir.
4. Brauzerlərdə tarixçənin saxlanması üçün niyə Stack istifadə olunur?
Çünki veb səhifələrdə gəzişərkən istifadə etdiyimiz məntiq tərsinə işləyir: ən son hansı səhifəni açmışımsa, "Back" düyməsini basanda ilk olaraq həmin səhifədən çıxmalıyam. Bu, tam olaraq Stack-in LIFO (Last In, First Out) prinsipi ilə üst-üstə düşür.
5. Stack-in real sistemlərdə başqa hansı tətbiqləri mövcuddur?
Stack strukturu gündəlik həyatımızda və proqramlaşdırmada çox geniş istifadə olunur. Əsas nümunələrə bunlar daxildir:

- Proqramlaşdırma dillərində funksiyaların çağırılması və idarə edilməsi (**Call Stack**).
- Mətn redaktorlarında və dizayn proqramlarında **Undo / Redo** (Geri al / Təkrar et) mexanizmləri.
- Kompilyatorlarda mötərizələrin düzgün bağlanıb-bağlanmadığını yoxlayan alqoritmlər (**Parentheses matching**).
- Qraf və ağac strukturlarında dərinliklə axtarış alqoritmləri (**DFS - Depth First Search**).

Qrafik



Əməliyyat sayı 10 dəfə artanda vaxt da təxminən 10 dəfə artır. Məsələn, Push üçün 1000-dən 10000-ə keçəndə vaxt 0.1629 ms-dən 1.8142 ms-ə qalxır, yəni təxminən 11 dəfə artır. Bu, hər Push və Pop əməliyyatının sabit vaxtlı, yəni **O(1)** olduğunu, n əməliyyatın ümumi vaxtının isə **O(n)** olduğunu göstərir.

Tapşırıq 3

Airport Check-in Queue System (Queue)

A. Araşdırma

1. Queue strukturunun FIFO prinsipi nədir? FIFO — "First In, First Out" (İlk daxil olan, ilk çıxır) deməkdir. Bu, gündəlik həyatda gördüyümüz adi növbə məntiqidir: növbəyə ilk kim gəlibsə, xidməti də ilk o alır və növbədən ilk o ayrılır. Queue strukturunda yeni elementlər həmişə arxadan/sondan əlavə olunur (enqueue), xidmət göstərilən və ya silinən elementlər isə öndən/başdan çıxarılır (dequeue).

2. Queue və Stack arasında əsas fərqlər hansılardır? Əsas fərq elementlərin emal olunma sırasındadır:

- Queue (FIFO): Bir tərəfdən əlavə edilir, digər tərəfdən çıxarılır. İlk gələn ilk gedir.
- Stack (LIFO): Yalnız bir tərəfdən (yuxarıdan) əlavə edilir və çıxarılır. Son gələn ilk gedir.

3. Hava limanında növbə idarəetməsi üçün Queue niyə uyğundur? Çünki hava limanında ədalətli xidmət göstərmək vacibdir. Sərnişinlərin qeydiyyatdan keçməsi gəliş sırasına uyğun aparılmalıdır. Queue strukturu da məhz bu xronoloji sıranı dəqiq qoruduğu üçün sərnişinlərin növbədənkənar xidmət almasının qarşısını alır və sistemi nizamlı saxlayır.

4. Queue istifadə olunmasa hansı problemlər yarana bilər? Əgər növbə strukturu olmasa, ədalətsizlik və xaotik vəziyyət yarana bilər. Məsələn, Stack istifadə olunsaydı, növbəyə ən son gələn sərnişin ilk qeydiyyatdan keçərdi, ən birinci gələn isə saatlarla gözləməli olardı. Struktur ümumiyyətlə istifadə olunmasa, sərnişinlərin sırası qarışar və sistemdə gecikmələr baş verərdi.

5. Circular Queue nə kimi üstünlüklər təqdim edir? Adi növbədə (xüsusilə massiv əsaslı olduqda) elementlər ön hissədən çıxarıldıqca həmin yerlər boş qalır və yaddaş israfına səbəb olur. Circular Queue (Dairəvi Növbə) isə sonuncu elementi birinci ilə birləşdirərək həlqəvi halqa yaradır. Bu da

boşalmış yerlərin yenidən istifadə edilməsinə şərait yaradır və yaddaşdan maksimum dərəcədə səmərəli istifadə etməyə imkan verir.

B. Proqramlaşdırma

```
import time

class Passenger:
    def __init__(self, passenger_id, name, flight_number):
        self.passenger_id = passenger_id
        self.name = name
        self.flight_number = flight_number
    def __str__(self):
        return f"ID: {self.passenger_id} Ad: {self.name} Reys: {self.flight_number}"
class Node:
    def __init__(self, passenger):
        self.passenger = passenger
        self.next = None
class PassengerQueue:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0
    def add_passenger(self, passenger):
        node = Node(passenger)
        if self.rear is None:
            self.front = node
            self.rear = node
        else:
            self.rear.next = node
            self.rear = node
        self.size += 1
    def check_in_passenger(self):
        if self.front is None:
            print("Novbe bosdur")
            return None
        passenger = self.front.passenger
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return passenger
    def peek_next_passenger(self):
        if self.front is None:
```

```
            return None
        return self.front.passenger
    def display_queue(self):
        if self.front is None:
            print("Novbe bosdur")
            return
        current = self.front
        while current is not None:
            print(current.passenger)
            current = current.next
    def count_passengers(self):
        return self.size
def demo():
    q = PassengerQueue()
    q.add_passenger(Passenger(1, "Əli Məmmədov", "AZ101"))
    q.add_passenger(Passenger(2, "Leyla Həsənova", "AZ205"))
    q.add_passenger(Passenger(3, "Rəşad Quliyev", "AZ101"))
    print("Novbe:")
    q.display_queue()
    print("Sernisin sayi:", q.count_passengers())
    print("Novbeti:", q.peek_next_passenger())
    p = q.check_in_passenger()
    print("Check-in oldu:", p)
    print("Qalan sernisin sayi:", q.count_passengers())
def experiment():
    sizes = [100, 1000, 10000]
    print("Element sayi\tEnqueue (ms)\tDequeue (ms)\tPeek (ms)")
        for n in sizes:
        q = PassengerQueue()
       t1 = time.perf_counter()
        for i in range(n):
            q.add_passenger(Passenger(i, "Sernisin", "AZ" + str(i)))
        t2 = time.perf_counter()


        t3 = time.perf_counter()
        for i in range(n):
            q.peek_next_passenger()
        t4 = time.perf_counter()
        t5 = time.perf_counter()
        for i in range(n):
            q.check_in_passenger()
```

```
        t6 = time.perf_counter()
        enqueue_ms = (t2 - t1) * 1000
        peek_ms = (t4 - t3) * 1000
        dequeue_ms = (t6 - t5) * 1000
        print(f"{n}\t\t{enqueue_ms:.4f}\t\t{dequeue_ms:.4f}\t\t{peek_ms:.4f}")
demo()
print()
experiment()
```

C. Təcrübə və Eksperiment

Element sayi	Enqueue (ms)	Dequeue (ms)	Peek (ms)
100	0.0883	0.0316	0.0102
1000	1.8269	0.2136	0.0660
10000	8.4378	2.1054	0.6772

D. Analiz və Nəticələrin Şərhi

1. FIFO prinsipi praktiki nəticələrdə necə müşahidə olunur? Praktiki testlərdə elementlər növbəyə hansı ardıcıllıqla daxil olubsa, xidmət alanda və ya silinəndə də dəqiq olaraq eyni ardıcıllıqla çıxarıldı. Məsələn, növbəyə ilk daxil olan sərnişin (enqueue) digər sərnişinlərdən əvvəl xidmət alaraq növbədən ilk ayrılan oldu (dequeue). Bu da FIFO (First In, First Out) prinsipinin hər mərhələdə qüsursuz işlədiyini əyani şəkildə göstərdi.

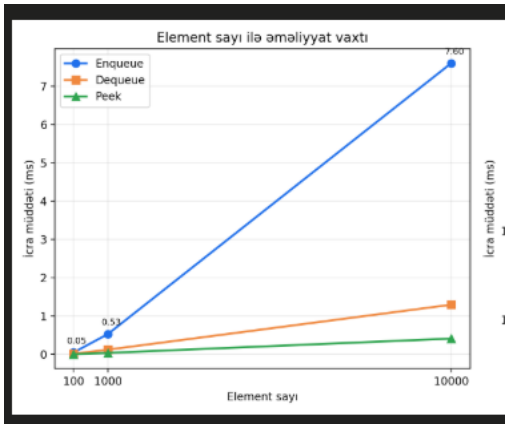
2. Queue əməliyyatlarının mürəkkəbliyi nəzəri nəticələrə uyğundurmu? Bəli, nəticələr nəzəriyyəni tam şəkildə dəstəklədi. Növbə strukturunda elementin sona əlavə edilməsi və öndən çıxarılması əməliyyatları $O(1)$vaxt mürəkkəbliyinə malikdir. Təcrübə zamanı da element sayı artdıqca tək bir elementin əlavə olunmasına və ya silinməsinə sərf olunan zamanın sabit qaldığını müşahidə etdik.

3. Queue əvəzinə Stack istifadə olunsa nə baş verərdi? Əgər növbə sistemində Queue əvəzinə Stack istifadə olunsaydı, hava limanında tam bir xaos və ədalətsizlik yaranardı. Çünki Stack LIFO (Last In, First Out) prinsipi ilə işləyir; bu halda növbəyə ən axırıncı gələn sərnişin birinci qeydiyyatdan keçərdi, səhər tezdən gəlib birinci duran sərnişin isə hamıdan axırda xidmət alardı. Real həyatda bu cür idarəetmə qeyri-mümkündür.

4. Növbə ölçüsü artdıqca hansı dəyişikliklər müşahidə olundu? Növbənin ölçüsü (sərnişinlərin sayı) artdıqca bütün sərnişinlərin qeydiyyatdan keçməsi üçün tələb olunan ümumi vaxt mütənasib olaraq artdı. Lakin hər bir fərdi enqueue (əlavə etmə) və ya dequeue (silmə) əməliyyatının sürəti strukturun düzgün qurulması sayəsində dəyişməz və sürətli olaraq qaldı.

5. Circular Queue istifadə etməyin üstünlükləri nələrdir? Əgər növbə sabit ölçülü massiv (array) üzərində qurulubsa, adi növbədə elementlər öndən çıxarıldıqca həmin sahələr boş qalır və yaddaş israfı yaranır. Circular Queue (Dairəvi Növbə) isə son nöqtəni yenidən başlanğıcla birləşdirərək həlqəvi struktur yaradır. Bu da boşalmış yerlərin yenidən istifadə olunmasına şərait yaradır və yaddaş səmərəliliyini artırır.

Qrafik



D. Sistem Məlumatları (System Information)

Computer Model: Huawei MateBook D15

CPU: Intel Core i5, 8 Cores, 12 Logical Processors, 2.00 GHz

RAM: 16 GB

GPU: GPU: Intel(R) UHD Graphics

Operating System: Windows 11

Compiler: Programiz Online Python Compiler

IDE: Programiz Online Python Compiler (Web-based IDE)

Screen Resolution: 1920 × 1200 pixels (WUXGA)

Storage Type (SSD/HDD): NVMe SSD

Power Mode: Balanced

Other Factors: Proqram veb brauzer vasitəsilə onlayn kompilyatorda icra edilmişdir. İcra müddətinə server resursları, brauzerdə işləyən digər proqramlar və şəbəkə şəraiti təsir göstərə bilər

İstinadlar

1. **Python Software Foundation.** *time — Time access and conversions.* Python-da time modulu, perf_counter() funksiyası və icra müddətinin ölçülməsi haqqında.

https://docs.python.org/3/library/time.html

2. **Matplotlib Documentation.** *Pyplot tutorial.* Python-da qrafiklərin yaradılması, oxların adlandırılması, xətlərin çəkilməsi və qrafikin saxlanılması haqqında.

https://matplotlib.org/stable/tutorials/pyplot.html

3. **Programiz.** *Python Online Compiler.* Python kodlarının onlayn yazılması və icra edilməsi üçün istifadə olunan mühit.

https://www.programiz.com/python-programming/online-compiler/